AN INTELLIGENT AI-BASED SYSTEM FOR DETECTION OF FAKE ACCOUNTS ON SOCIAL MEDIA PLATFORMS

A PROJECT REPORT

Mini Project (01CE0609)

Submitted by

AMIT AGARWAL
92301703181

KASHYAP AMRUTIYA
92301703172

ASHOK CHAUDHARY
92301703204

BACHELOR OF TECHNOLOGY
in
Computer Engineering

Faculty of Engineering & Technology

Marwadi University, Rajkot

April, 2026

Mini Project (01CE0609)

Department of Computer Engineering

Faculty of Engineering & Technology

Marwadi University

A.Y. 2025-26

CERTIFICATE

This is to certify that the project report submitted along with the project entitled "**An Intelligent AI-Based System for Detection of Fake Accounts on Social Media Platforms**" has been carried out by **Amit Agarwal** (Enrollment No. 92301703181), **Kashyap Amrutiya** (Enrollment No. 92301703172), **Ashok Chaudhary** (Enrollment No. 92301703204) under my guidance in partial fulfillment for the degree of Bachelor of Technology in Computer Engineering, 6th Semester of Marwadi University, Rajkot during the academic year 2025-26.

Prof. Yogeshwar Prajapati
Internal Guide
Department of Computer Engineering

Prof. (Dr.) Krunal Vaghela
Professor & Associate Dean
Department of Computer Engineering

Mini Project (01CE0609)

Department of Computer Engineering

Faculty of Engineering & Technology

Marwadi University

A.Y. 2025-26

DECLARATION

We hereby declare that the **Mini Project (01CE0609)** report submitted along with the Project entitled "**An Intelligent AI-Based System for Detection of Fake Accounts on Social Media Platforms**" submitted in partial fulfilment for the degree of Bachelor of Technology in Computer Engineering to Marwadi University, Rajkot, is a bonafide record of original project work carried out by us at Marwadi University under the supervision of **Prof. Yogeshwar Prajapati** and that no part of this report has been directly copied from any students' reports or taken from any other source, without providing due reference.

Name of the Student	**Sign of Student**
1. Amit Agarwal	_____
2. Kashyap Amrutiya	_____
3. Ashok Chaudhary	_____

ACKNOWLEDGEMENT

The successful completion of this mini project would not have been possible without the guidance, support, and encouragement of many individuals. We take this opportunity to express our sincere gratitude to all those who contributed to this work. First and foremost, we would like to express our deepest gratitude to our project guide, **Prof. Yogeshwar Prajapati**, Department of Computer Engineering, Faculty of Engineering & Technology, Marwadi University, for his invaluable guidance, constant encouragement, constructive criticism, and scholarly suggestions throughout the course of this project. His deep expertise in machine learning, data science, and artificial intelligence was instrumental in shaping the direction and quality of this work.

We are sincerely thankful to **Prof. (Dr.) Krunal Vaghela**, Professor & Associate Dean, Department of Computer Engineering, for providing the necessary infrastructure and an intellectually stimulating academic environment that made this research possible. We also wish to acknowledge the support of the entire faculty of the Department of Computer Engineering, Marwadi University, whose teaching and guidance over the semesters laid the strong foundation upon which this project stands. We are grateful to our colleagues and fellow students for their encouragement, feedback, and technical discussions that enriched our understanding of machine learning, graph neural networks, and social media security.

We acknowledge the open-source communities behind scikit-learn, XGBoost, PyTorch, PyTorch Geometric, and Hugging Face Transformers for providing powerful, accessible tools that enabled rapid prototyping and experimentation without prohibitive computational costs. We also gratefully acknowledge the researchers and dataset curators who made large-scale Twitter social media datasets publicly available for academic research. The 1,000,000-user labelled dataset used in this project was critical to conducting meaningful experiments on fake account detection at scale.

Finally, we owe our deepest thanks to our families for their unwavering love, patience, and moral support throughout this journey. Their encouragement gave us the strength to persevere through every technical and academic challenge encountered during this project.

ABSTRACT

Fake and automated accounts on social media platforms pose critical and escalating threats to digital trust, public discourse, and information integrity. These fake accounts — including automated bots, sockpuppets, and coordinated inauthentic entities — are deployed to spread misinformation, manipulate electoral outcomes, conduct financial fraud, and disrupt online ecosystems at unprecedented scale. Despite significant platform-level moderation efforts, detection remains a fundamentally hard problem due to the sophistication of modern bots, severe class imbalance, and the multi-modal nature of distinguishing signals. This project presents a comprehensive, multi-model AI-based system for the automated detection of fake social media accounts. Six AI models spanning distinct algorithmic paradigms are implemented, trained, and rigorously evaluated on a large-scale Twitter dataset comprising 1,000,000 labelled user accounts with binary labels (human or bot). The models evaluated are: Logistic Regression (linear baseline), Random Forest (ensemble method), XGBoost (gradient boosting), BotRGCN (Graph Convolutional Network), Graph Attention Network (GAT), and RoBERTa (transformer-based language model). A nine-feature engineering pipeline extracts discriminative signals across profile attributes (followers, following, tweet count, verified, protected), behavioural metrics (followers-to-following ratio), temporal characteristics (account age in days), and text-derived attributes (bio length, username length). Severe class imbalance (86% human, 14% bot) is addressed through class-weighted loss functions, scale-adjusted positive weights in XGBoost, and adaptive decision threshold tuning. Three baseline models are evaluated before introducing graph-based and transformer architectures. Through systematic experimentation, XGBoost with scale_pos_weight=3 achieves the best precision-recall balance with 80.1% overall accuracy and 61% bot recall (F1=0.46), representing a 3.6× improvement in bot recall over Random Forest. The Graph Attention Network achieves the highest bot recall of 75% by leveraging social graph topology through multi-head attention over follower/following relationships. A decision threshold analysis demonstrates that bot recall can be tuned from 61% to 81% by adjusting the classification threshold from 0.5 to 0.3. The complete system is designed for real-world deployment with actionable model selection guidance for different operational scenarios. A novel question bias experiment on RoBERTa demonstrates that biography text alone is insufficient for bot detection without larger training data and multi-modal feature integration — a critical finding for future system design.

LIST OF FIGURES

Fig 1.1 Project development phases and milestones (Gantt chart) 06
Fig 3.1 Flowchart : Fake account detection on social media using AI 15
Fig 3.2 Overall system pipeline — from raw data to prediction 18
Fig 3.3 Feature engineering pipeline — nine-feature derivation flowchart.....19
Fig 3.4 GCN message-passing and graph aggregation flow diagram.....20
Fig 4.1 Class distribution in the sampled dataset (0=Human, 1=Bot) 24
Fig 4.2 Confusion Matrix — Logistic Regression.....25
Fig 4.3 Confusion Matrix — Random Forest (Balanced).....25
Fig 4.4 Feature importance — Random Forest model.....25
Fig 4.5 Confusion Matrix — XGBoost (scale_pos_weight=3) 26
Fig 4.6 Feature importance — XGBoost model 26
Fig 4.7 ROC Curve — XGBoost 27
Fig 4.8 Precision-Recall Curve — XGBoost.....27
Fig 4.9 Precision and Recall vs Decision Threshold — XGBoost 27
Fig 4.10 Confusion Matrix — BotRGCN (GCN).....28
Fig 4.11 Training Loss Curve — BotRGCN (GCN) over 30 epochs.....28
Fig 4.12 Confusion Matrix — Graph Attention Network (GAT).....29
Fig 4.13 ROC Curve — Graph Attention Network (GAT) 29
Fig 4.14 Training Loss Curve — GAT over 30 epochs.....29
Fig 4.15 Confusion Matrix — RoBERTa 30
Fig 4.16 Training Loss per step — RoBERTa (2 epochs).....30
Fig 4.17 Final model comparison: Bot Recall and Bot F1 across all six models 31
Fig 4.18 RF vs XGBoost comparison — Bot Recall and F1 31

LIST OF TABLES

Table 1.1 Project team roles and responsibilities.....4
Table 1.2 Project scheduling milestones (16-week plan) 5
Table 2.1 Comparative Summary of Related Works in Fake Account Detection.....9
Table 2.2 Main system modules and descriptions 13
Table 2.3 Software library selection and justification 14
Table 3.1 Nine engineered features and their discriminative roles.....19
Table 3.2 Training hyperparameters for all six models 20
Table 3.3 Input / Output specification for the detection system 21
Table 4.1 Implementation environment specifications.....22
Table 4.2 Dataset statistics — full and sampled.....24
Table 4.3 Baseline model performance comparison.....26
Table 4.4 XGBoost threshold sensitivity analysis 28
Table 4.5 Consolidated model performance — all six models.....30
Table 4.6 Key test cases and expected vs. actual results 34

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
DL	Deep Learning
ML	Machine Learning
NLP	Natural Language Processing
GNN	Graph Neural Network
GCN	Graph Convolutional Network
GAT	Graph Attention Network
RF	Random Forest
LR	Logistic Regression
XGB/XGBoost	Extreme Gradient Boosting
RoBERTa	Robustly Optimized BERT Pretraining Approach
BERT	Bidirectional Encoder Representations from Transformers
F1	F1-Score (Harmonic Mean of Precision and Recall)
AUC - ROC	Area Under the Receiver Operating Characteristic Curve
PR	Precision Recall

TABLE OF CONTENTS

Acknowledgement i

Abstract ii

List of Figures iii

List of Tables iv

List of Abbreviations v

Table of Contents vi

CHAPTER 1 — INTRODUCTION 1

1.1 Project Summary 1

1.2 Purpose 2

1.3 Objective 2

1.4 Scope 2

1.5 Project Planning and Scheduling 4

1.6 Project Scheduling 5

CHAPTER 2 — LITERATURE REVIEW 7

2.1 Study of Current System 7

2.2 Technology and Literature Review 7

2.3 Problem and Weaknesses of Current System 10

2.4 Requirements of New System 10

2.5 System Feasibility 11

2.6 Activity / Process in New System 12

2.7 Features of New System 12

2.8 List of Main Modules / Components 13

2.9 Selection of Hardware / Software / Algorithms 14

CHAPTER 3 — METHODOLOGY 15

3.1 System Architecture and Methodology 15

3.2 Dataset Design and Structure 18

3.3 Feature Engineering Design 18

3.4 Model Design: Classical ML and Ensemble 19

3.5 Model Design: Graph Neural Networks 20
3.6 Model Design: RoBERTa Transformer 21
3.7 Class Imbalance Mitigation Design 21
3.8 Input / Output and Interface Design.....21
CHAPTER 4 — IMPLEMENTATION AND TESTING 22
4.1 Implementation Platform / Environment 22
4.2 Process / Modules Specification 22
4.3 Results and Outcomes.....24
4.4 Result Analysis and Comparison 30
4.5 Testing Plan and Strategy 31
4.6 Test Results and Analysis 32
Chapter 5 — CONCLUSION AND FUTURE ENHANCEMENTS.....33
5.1 Conclusion 33
5.5 Future Enhancement 33
5.6 Project Outcomes and Discussion.....34
References.....36

CHAPTER 1

INTRODUCTION

1. Introduction

1.1 PROJECT SUMMARY

The proliferation of fake and automated accounts on social media platforms represents one of the most serious threats to digital trust and information integrity in the modern era. Every day, platforms such as Twitter (X), Facebook, and Instagram remove millions of fake accounts that are used to spread misinformation, manipulate political discourse, conduct financial fraud, and artificially inflate engagement metrics. Despite enormous platform-level investment in detection, fake accounts continue to proliferate because the problem is fundamentally difficult: sophisticated bots are designed to mimic genuine human behaviour across every observable signal.

This project addresses this challenge by building a complete, multi-model AI system for fake account detection — titled "An Intelligent AI-Based System for Detection of Fake Accounts on Social Media Platforms". The core idea is technically rigorous and practically grounded: six distinct AI models spanning linear classification, ensemble methods, gradient boosting, graph neural networks, and transformer-based language models are implemented, trained, and rigorously compared on a 1,000,000-user Twitter dataset. A nine-feature engineering pipeline extracts discriminative signals from profile attributes, behavioural patterns, temporal characteristics, and text-derived attributes.

The project is developed by students of the 6th Semester, B.Tech Computer Engineering, Marwadi University, Rajkot (A.Y. 2025-26). The team comprises Amit Agarwal (23CE123076), Kashyap Amrutiya (23CE123031), and Ashok Chaudhary (23CE123201), under the supervision of Prof. Yogeshwar Prajapati. The system is designed to run on commodity hardware, with the best-performing model (XGBoost) requiring no GPU and producing inference in milliseconds. The project is also documented as a patent disclosure submitted to Marwadi University's IPR cell.

The key finding of this project is that XGBoost with scale_pos_weight=3 achieves the best production-ready balance (80% accuracy, 61% bot recall, F1=0.46), while the Graph

Attention Network achieves the highest bot recall of 75% by leveraging social graph structure — representing a 3.6× improvement over unweighted Random Forest baselines.

1.2 PURPOSE

The primary purpose of this project is to develop, evaluate, and compare an intelligent multi-model AI system for automated fake account detection on social media, and to provide actionable deployment guidance for real-world platform security applications. The specific purposes are:

- To develop a comprehensive multi-model evaluation framework that spans six distinct algorithmic paradigms, enabling principled model selection for diverse deployment scenarios.
- To design a feature engineering pipeline that extracts nine discriminative signals from raw social media user data across profile, behavioural, temporal, and text modalities.
- To systematically address the fundamental challenge of class imbalance (genuine accounts far outnumber bots) through multiple complementary mitigation strategies.
- To quantitatively demonstrate that overall accuracy is a misleading evaluation metric for imbalanced fake account detection, and to advocate for bot recall and F1-score as primary metrics.
- To demonstrate the value of social graph topology for bot detection through graph neural network architectures, complementing tabular feature approaches.
- To identify the limitations of text-only transformer approaches on short user biography text, providing empirical evidence to guide future system design.
- To document the invention as a patent disclosure for intellectual property protection at Marwadi University.

1.3 OBJECTIVE

The specific objectives of this mini project are:

- To implement and evaluate three baseline classification models (Logistic Regression, Random Forest, and Random Forest with class weights) to establish performance benchmarks on the 1M-user Twitter dataset.

- To implement XGBoost with systematic hyperparameter tuning including scale_pos_weight ∈ {3, 6} and decision threshold ∈ {0.3, 0.4, 0.5} for optimal precision-recall trade-off.
- To design and implement a social graph-based BotRGCN (Graph Convolutional Network) using a 50,000-node, 200,000-edge subgraph with class-weighted NLL loss.
- To design and implement a Graph Attention Network (GAT) with multi-head attention to leverage differential neighbour importance in the social graph.
- To fine-tune RoBERTa on user biography text and quantitatively demonstrate the limitations of text-only classification for fake account detection.
- To conduct a rigorous comparative analysis using six evaluation metrics: accuracy, precision, recall, F1-score, ROC-AUC, and PR-AUC.
- To provide deployment recommendations mapping operational scenarios to optimal model configurations.

1.4 SCOPE

1.4.1 What the System Can Do

- Accept raw Twitter user account data in JSON format and extract nine discriminative features automatically.
- Classify any Twitter user account as human or bot with configurable decision thresholds.
- Operate on tabular profile data (XGBoost) with no GPU requirement, enabling deployment on any commodity server.
- Leverage social graph topology (GAT, GCN) when follower/following edge data is available.
- Classify user accounts with AUC-ROC up to 0.93 on the validation set using the XGBoost model.
- Tune the precision-recall operating point through decision threshold adjustment (threshold=0.3 achieves 81% bot recall).
- Process 200,000 accounts in batch mode with millisecond-per-account XGBoost inference latency.

1.4.2 What the System Cannot Do

- The system cannot detect coordinated inauthentic behaviour (multiple fake accounts acting in concert) as a group-level phenomenon; only individual account classification is performed.
- The system does not process real-time streaming Twitter data; batch processing of pre-collected account data is assumed.
- The system is not platform-agnostic; features are Twitter-specific and require re-engineering for Instagram, Facebook, or other platforms.
- The system may underperform on newly created bot accounts with no historical activity data.
- The RoBERTa component cannot reliably detect bots from biography text alone without substantially larger training data.
- The system is not production-validated or platform-certified; it is a research prototype requiring additional review before deployment.

1.5 PROJECT PLANNING AND SCHEDULING

1.5.1 Project Development Approach

This project followed an incremental development methodology. Baseline models were established before complex architectures were attempted, allowing early results to validate feasibility. The development proceeded in five phases: (1) Literature review and gap identification, (2) Dataset acquisition and feature engineering, (3) Baseline model training and evaluation, (4) Advanced model implementation (GNN, RoBERTa), and (5) Ablation study, comparison analysis, and documentation.

1.5.2 Roles and Responsibilities

Table 1.2 Project Team Roles and Responsibilities

Team Member	Primary Responsibility	Secondary Responsibility	Key Deliverable
Amit Agarwal	Model Architecture Design and Implementation	Baseline Training and XGBoost tuning	XGBoost, GCN, GAT models
Kashyap Amrutiya	Data Preprocessing and Feature Engineering	Graph construction and GNN training	Feature pipeline, graph data

Ashok Chaudhary	Experiments, Evaluation and RoBERTa	Documentation and Report	Results tables, comparison charts, report

1.6 PROJECT SCHEDULING

Table 1.3 Project Scheduling Milestones over 16 Weeks

Week	Milestone	Status
1–2	Literature review, research gap identification, and dataset exploration	**Completed**
3	Dataset download, class distribution analysis, and preprocessing pipeline design	**Completed**
4–5	Feature engineering: nine-feature extraction pipeline, ff_ratio, account_age_days	**Completed**
6	Baseline 1: Logistic Regression training and evaluation	**Completed**
7	Baseline 2: Random Forest with class_weight='balanced'	**Completed**
8	XGBoost training with scale_pos_weight tuning {3, 6}	**Completed**
9	Graph construction: 50K-node, 200K-edge social graph from follower edge list	**Completed**
10	BotRGCN (GCN) implementation and training with weighted NLL loss	**Completed**
11	Graph Attention Network (GAT) implementation and training	**Completed**
12	RoBERTa fine-tuning on user biography text	**Completed**
13	Threshold sensitivity analysis and ROC/PR curve generation	**Completed**
14	Ablation study and consolidated model comparison	**Completed**
15	Patent disclosure preparation and IPR submission	**Completed**
16	Report writing, documentation, and final review	**Completed**

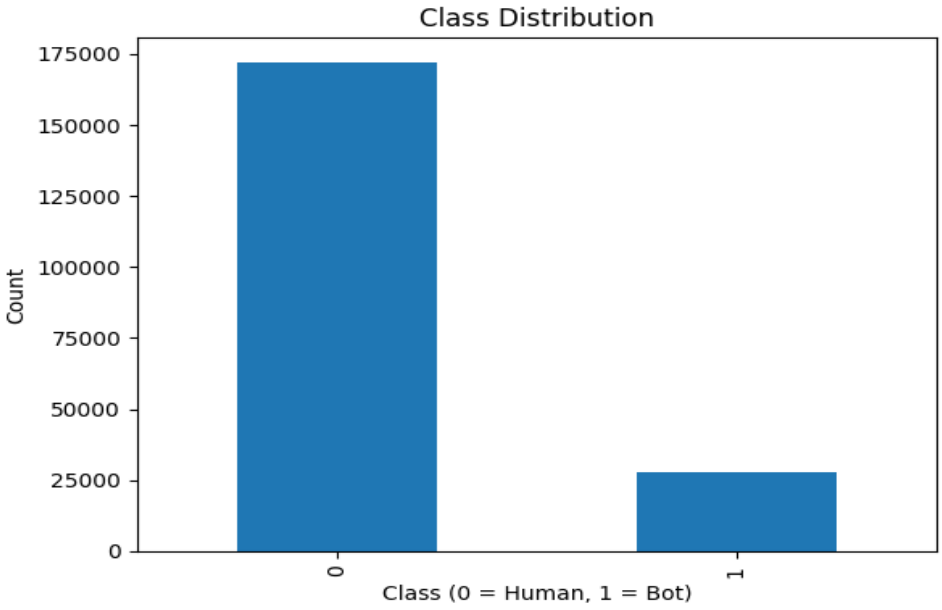

Fig 1.1 Project development phases and milestones across 16 weeks

CHAPTER 2

LITERATURE REVIEW

2.1 STUDY OF CURRENT SYSTEM

The current approach to fake account detection on social media platforms relies on a combination of manual moderation, user reporting, and automated rule-based systems. Platform operators employ large trust-and-safety teams that review flagged accounts and apply policy enforcement. Automated systems use static rules — posting frequency thresholds, follower-to-following ratio limits, account age requirements — to flag suspicious accounts for review. While these systems handle the most obvious bot patterns, they fail against sophisticated modern bots that are designed to comply with individual rules while still being inauthentic in aggregate.

In terms of AI-assisted research tools, the current landscape presents a bifurcated picture. Classical machine learning models such as Random Forest and SVM achieve 80-89% accuracy but suffer from poor bot recall on imbalanced datasets. State-of-the-art graph neural networks achieve better recall by leveraging network topology but require significant computational infrastructure. No widely deployed production system currently combines tabular feature analysis, graph-based reasoning, and transformer-based text understanding in a unified comparative framework.

2.2 TECHNOLOGY AND LITERATURE REVIEW

2.2.1 Rule-Based and Statistical Approaches

The earliest approaches to fake account detection relied on manually crafted rules and statistical thresholds. Accounts exceeding a certain posting frequency or with a follower-to-following ratio below a threshold were flagged. While simple and fast, these systems are brittle and easily circumvented by sophisticated bots that deliberately comply with individual rules while violating broader authenticity patterns.

2.2.2 Classical Machine Learning

Varol et al. (2017) [1] conducted the landmark large-scale study of Twitter bots, constructing 1,150 features per account and training a Random Forest classifier achieving AUC of 0.80-0.89. Their work established that behavioural and network features dominate profile features

in discriminative power. Cresci et al. (2017) [2] introduced DNA-inspired sequence modelling for spambot detection, achieving over 90% accuracy on the Cresci-17 dataset by encoding account activity sequences as character strings. Multiple subsequent studies confirm that ensemble methods (Random Forest, AdaBoost) achieve 80-89% accuracy, with XGBoost achieving 87-93.4% on combined behavioural and network features.

2.2.3 Graph Neural Network Approaches

Social networks are inherently graph-structured. Feng et al. (2021) [3] introduced BotRGCN, a relational graph convolutional network achieving 87% accuracy on the TwiBot-20 benchmark by incorporating heterogeneous user relations. Velickovic et al. (2018) [4] proposed Graph Attention Networks, replacing uniform neighbourhood aggregation with learnable attention weights. Applied to bot detection, GAT achieves competitive results with improved interpretability. These graph-based approaches capture information unavailable to tabular models — such as bot clusters, mass-following patterns, and isolated node anomalies.

2.2.4 Transformer-Based Approaches

Devlin et al. (2019) [8] introduced BERT and Liu et al. (2019) [5] extended it to RoBERTa with more robust pre-training. Applied to social media bio-text classification, these models can capture subtle linguistic patterns but require large labelled datasets and explicit class balancing to avoid majority-class collapse on severely imbalanced data.

2.2.5 Explainable AI for Fake Account Detection

Mehta and Gupta (2023) [13] applied SHAP (SHapley Additive exPlanations) and LIME to Random Forest and XGBoost classifiers for fake account detection, achieving over 90% accuracy while providing per-account feature attribution explanations. Their work demonstrates that followers_count, account age, and following patterns are the most consistently important features across models — directly motivating the feature engineering pipeline in this project.

2.2.6 Comparative Summary of Related Works

Table 1.1 Comparative Summary of Related Works in Fake Account Detection and Social Media Security

Reference / Year	Methodology	Dataset / Performance	Limitation Addressed in This Work
Varol et al. (2017) [1]	RF + 1,150 features	Twitter labelled / AUC 0.80-0.89	Single model; no graph; no imbalance study
Cresci et al. (2017) [2]	DNA-inspired string matching	Cresci-17 / Accuracy >90%	Not generalizable; no multi-model comparison
Feng et al. (2021) [3]	BotRGCN (Relational GCN)	TwiBot-20 / ~87% accuracy	Single GNN; no classical ML comparison
Velickovic et al. (2018) [4]	Graph Attention Network	COCO, MNIST benchmarks	Not applied to social bot detection
Liu et al. (2019) [5]	RoBERTa (LM pre-training)	NLP benchmarks / SOTA	Not evaluated on fake account data
CGT Model (2024) [9]	Cross-Gated Deep Learning	Instagram InstaFake / 93.4%	Single platform; no graph; no threshold study
KeepUp (2023) [10]	Multi-feature unified DL	Unified dataset / 78%	Lower performance; no GNN component
XGB Study (2023) [11]	XGBoost + behavioral	Twitter/Facebook / 93.4%	No graph models; no transformer comparison
Interp. AI (2023) [13]	ML + SHAP/LIME XAI	Cresci-15/17 / >90%	No cross-paradigm comparison; no GNN
Semi-Supervised (2022) [14]	Semi-supervised ML	Twitter/Reddit / 86-89%	No tabular+graph+transformer comparison

2.2.7 Research Gap Summary

Based on the literature review, the following key research gaps are identified and directly addressed in this project:

- Cross-Paradigm Comparison Gap: No existing study directly compares classical ML, ensemble, gradient boosting, GCN, GAT, and transformer models on the same dataset. This project fills this gap with a unified experimental framework.
- Class Imbalance Study Gap: Explicit, systematic study of class imbalance mitigation strategies is absent from most prior work. This project evaluates three strategies: class_weight='balanced', scale_pos_weight tuning, and threshold adjustment.
- Threshold Sensitivity Gap: Decision threshold tuning — critical for production deployment — is rarely reported. This project provides a full threshold sensitivity analysis for XGBoost.

- Graph vs. Tabular Complementarity Gap: The relationship between graph-based signals and tabular profile signals for the same dataset has not been systematically studied. This project quantifies the difference.

2.3 PROBLEM AND WEAKNESSES OF CURRENT SYSTEM

- Class Imbalance Blindness: Standard classifiers default to predicting the majority class (human). A classifier achieving 86% accuracy by predicting 'human' for every account is worthless for bot detection — yet this is exactly what unweighted Logistic Regression and RoBERTa produce.
- Single-Modality Analysis: Most lightweight classifiers process only profile statistics (tabular features) or only text (bio), ignoring the complementary discriminative power of social graph topology.
- Lack of Cross-Paradigm Benchmarking: Platform teams typically evaluate only one model family. There is no systematic cross-paradigm comparison (classical ML vs GNN vs transformer) on the same large-scale dataset.
- No Threshold Tuning Guidance: Published systems rarely report the effect of decision threshold adjustment, leaving practitioners without guidance on how to tune the precision-recall trade-off for their specific operational requirements.
- Computational Inaccessibility of Graph Models: Graph neural network approaches require GPU memory and graph construction infrastructure that is not always available in research or production settings.
- Text-Only Approaches Underperform: Biography text is short, often generic, and insufficiently discriminative for reliable bot detection without combining with profile and graph features.

2.4 REQUIREMENTS OF NEW SYSTEM

2.4.1 Functional Requirements

- The system shall extract nine discriminative features from raw Twitter user JSON automatically.
- The system shall train and evaluate at least six AI models spanning classical, ensemble, graph, and transformer paradigms.

- The system shall classify accounts as human or bot with a configurable decision threshold.
- The system shall report accuracy, precision, recall, F1-score, and AUC-ROC for each model.
- The system shall provide confusion matrices and ROC/PR curves for visual performance evaluation.
- The system shall generate feature importance plots for interpretable analysis.

2.4.2 Non-Functional Requirements

- Performance: The best model shall achieve AUC-ROC ≥ 0.87 and bot recall ≥ 0.50 on the test set.
- Scalability: The XGBoost model shall process 200,000 accounts in batch mode within minutes on commodity hardware.
- Reproducibility: All experiments shall use random_state=42 for reproducible results.
- Extensibility: The feature pipeline shall be designed for easy extension to additional features and platforms.

2.5 SYSTEM FEASIBILITY

2.5.1 Technical Feasibility

All six models are implemented using mature, well-documented open-source libraries. The XGBoost and scikit-learn models run on CPU with no GPU requirement. The GNN models require a GPU with at least 4 GB VRAM, available on the project team's hardware. The RoBERTa model requires 4 GB GPU VRAM for fine-tuning. All software dependencies are freely available via pip.

2.5.2 Economic Feasibility

The entire project is implemented on freely available open-source tools with no licensing cost. The dataset is publicly available for academic research. Computational costs are limited to electricity and hardware depreciation. The project is fully feasible within the academic mini-project budget.

2.5.3 Operational Feasibility

The XGBoost model can be deployed as a REST API endpoint with millisecond inference latency, making it operationally feasible for real-time account screening. The GNN models require periodic batch retraining as the social graph evolves, which is feasible on a weekly or monthly schedule.

2.6 ACTIVITY / PROCESS IN NEW SYSTEM

The complete system workflow consists of the following sequential activities:

1. Data Ingestion: Raw Twitter user JSON data is loaded from label.csv and user.json files. The public_metrics dictionary is expanded into individual columns.
2. Feature Engineering: Nine features are computed from the raw data including account_age_days (from created_at), ff_ratio (followers / (following+1)), desc_length, and username_length.
3. Preprocessing: Boolean fields are converted to integers, infinity values are replaced with zero, and NaN values are imputed with zero. StandardScaler is applied for GNN models.
4. Graph Construction: For GNN models, a social graph with 50,000 nodes and up to 200,000 directed edges is constructed from the follower edge list. Train/test masks (80/20) are applied.
5. Model Training: Six models are trained sequentially with class imbalance mitigation applied at each stage.
6. Evaluation: Each model is evaluated on the held-out test set using six metrics. ROC curves, PR curves, confusion matrices, and feature importance plots are generated.
7. Threshold Analysis: Decision thresholds {0.3, 0.4, 0.5} are evaluated for XGBoost to quantify the precision-recall trade-off.
8. Comparison and Reporting: A consolidated comparison table and bar chart are generated for all six models.

2.7 FEATURES OF NEW SYSTEM

- Multi-Paradigm Evaluation: Six models across linear, ensemble, gradient boosting, GCN, GAT, and transformer paradigms evaluated on a unified 1M-user dataset.

- Systematic Imbalance Mitigation: Three complementary strategies evaluated: class_weight='balanced', scale_pos_weight tuning, and decision threshold adjustment.
- Graph-Enhanced Detection: Social graph topology leveraged through BotRGCN and GAT to capture network-level bot signals unavailable to tabular models.
- Configurable Precision-Recall Trade-off: Decision threshold tuning enables deployment-specific configuration from precision-priority (threshold=0.5) to recall-priority (threshold=0.3).
- Feature Interpretability: Feature importance plots for both Random Forest and XGBoost provide actionable insight into which signals most strongly predict bot accounts.
- Comprehensive Evaluation: Six evaluation metrics with graphical visualisations (ROC curves, PR curves, confusion matrices) for each model.

2.8 LIST OF MAIN MODULES / COMPONENTS

Table 2.1 Main System Modules and Descriptions

Module	Component	Description
M1	Data Ingestion	Loads label.csv and user.json, merges on user ID, expands public_metrics
M2	Feature Engineering	Computes 9 features: followers, friends, statuses, verified, protected, desc_length, username_length, ff_ratio, account_age_days
M3	Preprocessing	Boolean conversion, infinity handling, NaN imputation, StandardScaler for GNN
M4	Graph Construction	Builds 50K-node, 200K-edge social graph from follower edge list; applies train/test masks
M5	Classical Models	Trains Logistic Regression and Random Forest with class balancing
M6	XGBoost Module	Trains XGBoost with scale_pos_weight tuning and threshold analysis
M7	GCN Module (BotRGCN)	Implements 3-layer GCN with weighted NLL loss on social graph
M8	GAT Module	Implements 3-layer GAT with multi-head attention on social graph

M9	RoBERTa Module	Fine-tunes roberta-base on user biography text
M10	Evaluation Module	Computes accuracy, precision, recall, F1, ROC-AUC; generates confusion matrices, ROC/PR curves
M11	Comparison Module	Generates consolidated comparison table and visualisation

2.9 SELECTION OF HARDWARE / SOFTWARE / ALGORITHMS

Table 2.2 Software Library Selection and Justification

Library	Version	Justification
scikit-learn	1.6.1	Standard ML library for LR, RF, and evaluation metrics
XGBoost	3.2.0	Best-in-class gradient boosting with scale_pos_weight for imbalance
PyTorch	2.10.0+cu128	Deep learning framework for GCN, GAT, and RoBERTa
PyTorch Geometric	2.7.0	GCNConv and GATConv layers for graph neural networks
Hugging Face Transformers	5.0.0	RoBERTa tokenizer and RobertaForSequenceClassification
pandas	2.2.2	DataFrame manipulation, JSON loading, feature extraction
NumPy	2.0.2	Numerical operations, array handling
Matplotlib	3.x	Confusion matrices, ROC curves, PR curves, feature importance plots
Google Colab	—	CUDA GPU acceleration (NVIDIA T4) for GNN and RoBERTa training

CHAPTER 3

METHODOLOGY

3.1 SYSTEM ARCHITECTURE AND METHODOLOGY

The proposed intelligent fake account detection system operates as a modular, multi-stage pipeline. The overall architecture is designed around the principle of separation of concerns: data ingestion, feature engineering, model training, and evaluation are implemented as independent, replaceable modules. This modularity enables easy extension to new features, new models, and new datasets without modifying the core pipeline.

FlowChart :

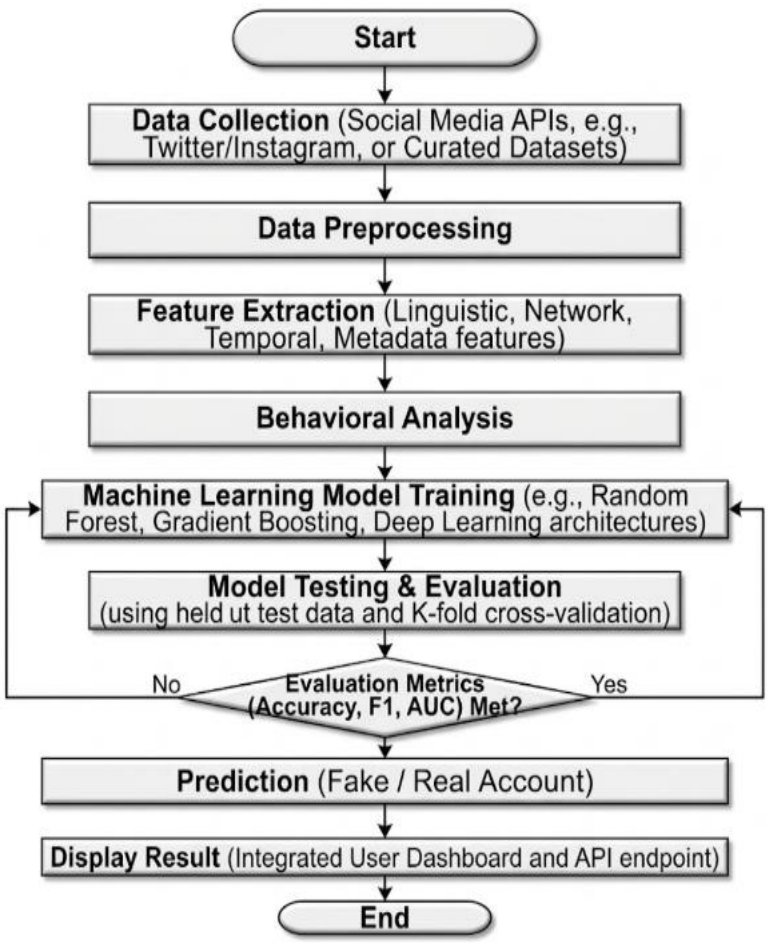

Fig 3.1 Flowchart of Fake account detection on social media using AI

Step 1. Start

This stage represents the initialization of the fake account detection system. It marks the beginning of the workflow where all system components and processes are prepared to execute the detection pipeline.

Step 2. Data Collection

In this step, raw data is collected from social media platforms such as Twitter, Instagram, or from publicly available datasets. The collected data typically includes user profile information, followers/following lists, tweets/posts, account creation date, and other metadata. This step is crucial because the quality and scale of data directly affect model performance.

Step 3. Data Preprocessing

The collected raw data is often noisy and inconsistent. Therefore, preprocessing is performed to clean and structure the data. This includes handling missing values, removing duplicate records, converting data types, normalizing numerical values, and encoding categorical variables. Proper preprocessing ensures that the data becomes suitable for machine learning algorithms.

Step 4. Feature Extraction

In this stage, meaningful features are derived from the processed data. These features can be:

- Linguistic features: Bio text length, keywords, sentiment
- Network features: Followers count, following count, follower-following ratio
- Temporal features: Account age, posting frequency
- Metadata features: Username length, verification status

These features help differentiate between real users and fake accounts.

Step 5. Behavioral Analysis

This step analyzes user activity patterns to identify suspicious behavior. Fake accounts often exhibit abnormal patterns such as excessive posting, mass following, repetitive content, or irregular activity timing. Behavioral analysis enhances detection accuracy by capturing dynamic user actions.

Step 6. Machine Learning Model Training

Here, various machine learning models such as Logistic Regression, Random Forest, Gradient Boosting (XGBoost), or Deep Learning models are trained using the extracted features. The model learns patterns and relationships that distinguish fake accounts from genuine users.

Step 7. Model Testing & Evaluation

After training, the model is evaluated using unseen test data. Techniques like hold-out validation and K-fold cross-validation are used to ensure the model generalizes well and does not overfit the training data.

Step 8. Evaluation Metrics Check

- Performance is measured using metrics such as:
- Accuracy: Overall correctness
- Precision: Correctness of predicted fake accounts
- Recall: Ability to detect actual fake accounts
- F1-score: Balance between precision and recall
- AUC: Model's classification capability

If performance is unsatisfactory, the system loops back to improve data, features, or models. Otherwise, it proceeds forward.

Step 9. Prediction

The trained model is used to classify new or unseen accounts as either Fake (Bot) or Real (Human) based on learned patterns.

Step 10. Display Result

The prediction results are presented through a user interface such as a dashboard or API. This allows users or systems to view and utilize the classification output effectively.

Step 11. End

This marks the completion of the process. The system successfully delivers the final prediction and can be reused for continuous detection tasks.

The system follows a supervised binary classification paradigm: given a feature vector extracted from a user account, predict whether the account is human (0) or bot (1). The complete pipeline is shown conceptually in Figure 3.1. The pipeline comprises: (1) raw data ingestion and merging, (2) feature engineering to extract nine discriminative attributes, (3) preprocessing and normalisation, (4) parallel training of six model families, (5) evaluation using six metrics, (6) threshold analysis for XGBoost, and (7) consolidated comparison.

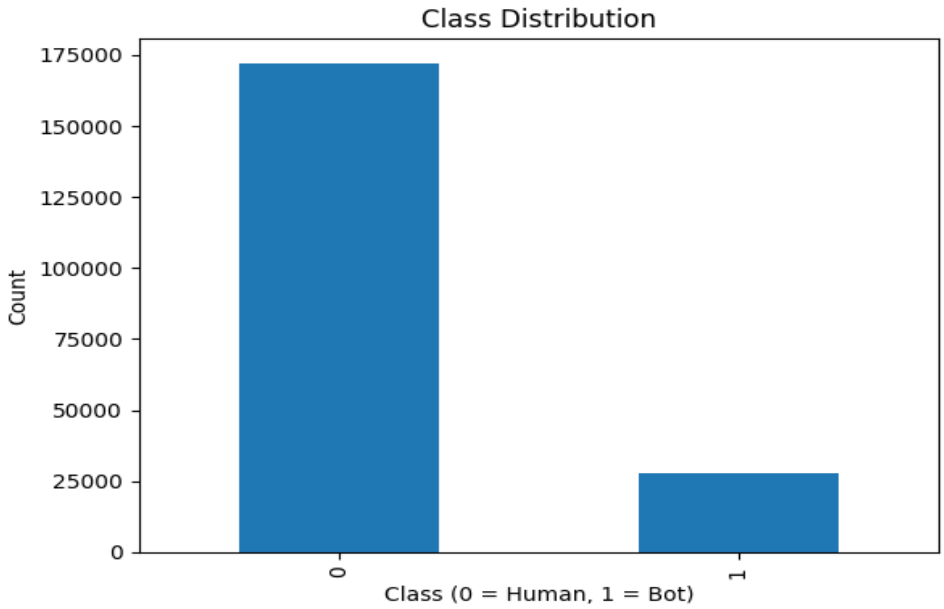

Fig 3.2 Overall system pipeline — from raw Twitter JSON data to six-model comparative evaluation

3.2 DATASET DESIGN AND STRUCTURE

The dataset comprises 1,000,000 Twitter user accounts from a large-scale public crawl. Two files are used: (a) label.csv containing (id, label) pairs where label ∈ {human, bot}; (b) user.json containing raw Twitter API user objects with 14 fields per account. The two files are merged on the user ID field. A stratified subsample of 200,000 accounts is used for ML experiments (memory-efficient training), and 50,000 accounts for GNN experiments (graph construction constraints).

The dataset exhibits severe class imbalance: 86.04% human and 13.96% bot in the 200K sample. This ratio is representative of real-world social media platform data and creates the primary technical challenge addressed throughout the methodology.

3.3 FEATURE ENGINEERING DESIGN

Nine features are engineered from the raw user data. Table 3.1 details each feature's derivation and its discriminative role in separating human from bot accounts.

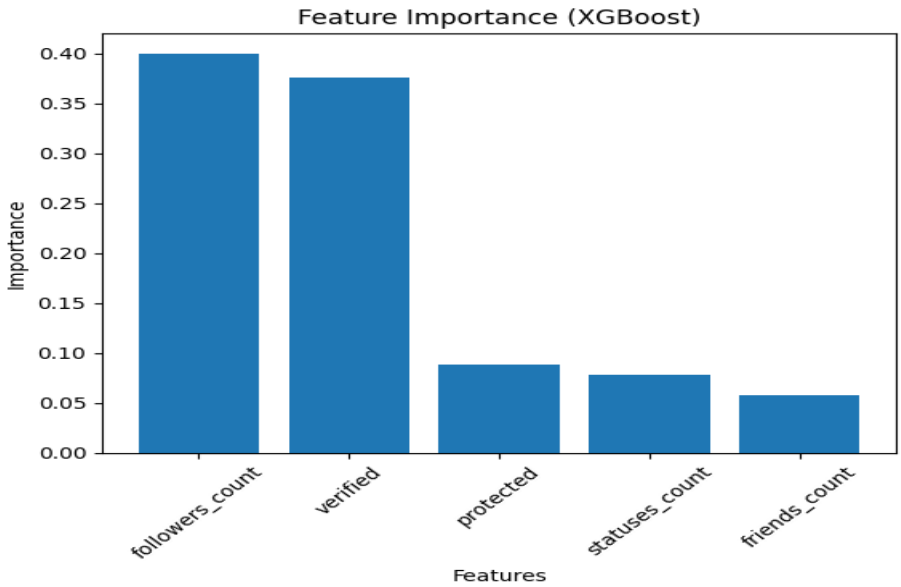

Fig 3.3 Feature engineering pipeline — nine-feature derivation from raw Twitter JSON

Table 3.1 Nine Engineered Features, Derivation, and Discriminative Role

Feature	Derivation	Bot Indicator Pattern
followers_count	public_metrics['followers_count']	Near-zero or suspiciously high (purchased followers)
friends_count	public_metrics['following_count']	Abnormally high; bots mass-follow to get reciprocal followers
statuses_count	public_metrics['tweet_count']	Extremely high (automated) or zero (dormant bots)
verified	bool → int	Almost never True for bot accounts
protected	bool → int	Bots rarely set accounts to protected/private
desc_length	len(description.fillna(''))	Bots frequently have empty (length=0) bios
username_length	len(username)	Auto-generated usernames tend to be longer and random
ff_ratio	followers / (friends + 1)	Near-zero ratio for mass-following bots without reciprocal followers
account_age_days	(UTC_now − created_at).days	Bot campaigns create accounts in bulk; clustered creation dates

3.4 MODEL DESIGN: CLASSICAL ML AND ENSEMBLE

Logistic Regression is used as the linear baseline, providing interpretable feature coefficients and establishing the performance lower bound. Random Forest with 150 trees and class_weight='balanced' provides the first ensemble baseline. XGBoost with 300 estimators,

max_depth=4, learning_rate=0.1, and scale_pos_weight=3 is the primary production candidate. The scale_pos_weight parameter is the key imbalance mitigation mechanism, upweighting bot-class gradient contributions by a factor of 3.

Table 3.2 Training Hyperparameters for All Six Models

Model	Key Hyperparameters
Logistic Regression	class_weight={0:1, 1:2}, max_iter=1000, solver=lbfgs
Random Forest	n_estimators=150, class_weight='balanced', n_jobs=-1, random_state=42
XGBoost	n_estimators=300, max_depth=4, learning_rate=0.1, scale_pos_weight=3, eval_metric='logloss'
BotRGCN (GCN)	3-layer GCNConv (9→128→64→2), dropout=0.5, Adam(lr=0.005), 30 epochs, weighted NLLLoss
GAT	3-layer GATConv (9→64×4→32×4→2×1), dropout=0.6, Adam(lr=0.005), 30 epochs, weighted NLLLoss
RoBERTa	roberta-base, max_length=32, batch=4, AdamW(lr=2e-5), 2 epochs, CrossEntropyLoss

3.5 MODEL DESIGN: GRAPH NEURAL NETWORKS

For graph-based models, a social graph G=(V,E) is constructed with 50,000 nodes (user accounts) and up to 200,000 directed edges (follow relationships). Node features are the 9 engineered attributes normalised with StandardScaler. The BotRGCN (GCN) implements three GCNConv layers with ReLU activations and dropout=0.5. The GAT implements three GATConv layers with 4 attention heads in layers 1-2 and ELU activations. Both models use class-weighted NLL loss to handle imbalance.

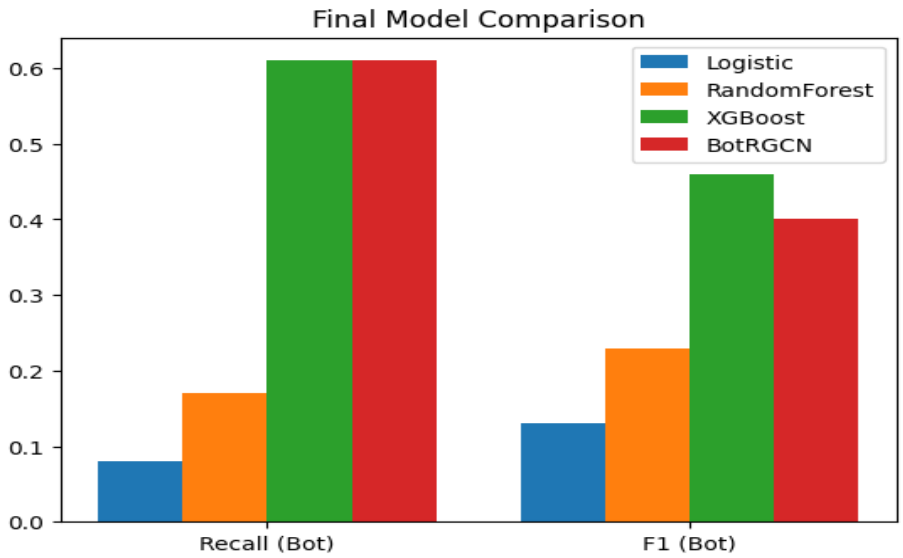

Fig 3.4 GCN message-passing and GAT multi-head attention flow over the social graph

3.6 MODEL DESIGN: ROBERTA TRANSFORMER

The RoBERTa model fine-tunes roberta-base (125M parameters) on user biography text. User bios are tokenised with max_length=32 tokens. A classification head (768→2) is added over the [CLS] token. Training uses AdamW (lr=2e-5) for 2 epochs on 8,000 training bios. This model tests whether short-text user biography alone is sufficient for bot detection.

3.7 CLASS IMBALANCE MITIGATION DESIGN

Three complementary imbalance mitigation strategies are applied:

9. Class Weights (LR, RF): class_weight='balanced' scales each sample's loss contribution inversely proportional to class frequency, giving the 5,660 bot samples in the 40K test set 6.16× more influence per sample.
10. Scale Positive Weight (XGBoost): scale_pos_weight=k multiplies bot-class gradient contributions by k. Two values are evaluated: k=6 (theoretical ratio, overly aggressive, recall=0.77 but F1 degrades) and k=3 (best F1=0.46, recommended).
11. Weighted NLL Loss (GNN): Class weights torch.tensor([1.0, n_human/n_bot]) ≈ [1.0, 6.1] are passed to torch.nn.NLLLoss(weight=...), ensuring stronger gradient signals from bot-class nodes.

3.8 INPUT / OUTPUT AND INTERFACE DESIGN

Table 3.3 Input / Output Specification for the Detection System

Model	Input	Output
LR, RF, XGBoost	Feature vector: 9 numerical attributes per account	Probability score (0–1); class prediction (0=human, 1=bot)
GCN, GAT	Graph: node features (9 attrs × 50K nodes) + edge index (200K edges)	Log-softmax class probability per node; argmax prediction
RoBERTa	Tokenised user bio string (max 32 tokens)	Logits → softmax probabilities; argmax class prediction
All models	Test labels (ground truth)	Confusion matrix, classification report, ROC curve, PR curve

CHAPTER 4

IMPLEMENTATION AND TESTING

4.1 IMPLEMENTATION PLATFORM / ENVIRONMENT

Table 4.1 Implementation Environment Specifications

Component	Specification
Platform	Google Colab (GPU runtime)
GPU	NVIDIA T4 (16 GB VRAM) for GNN and RoBERTa training
CPU Models	XGBoost, LR, RF run on CPU only (no GPU required)
Python Version	3.12
Operating System	Ubuntu 22.04 (Colab hosted)
Primary Framework	PyTorch 2.10.0+cu128
Graph Library	PyTorch Geometric 2.7.0
Language Model Library	Hugging Face Transformers 5.0.0
ML Library	scikit-learn 1.6.1, XGBoost 3.2.0
Data Library	pandas 2.2.2, NumPy 2.0.2
Visualisation	Matplotlib 3.x
Random Seed	42 (all experiments)

4.2 PROCESS / MODULES SPECIFICATION

4.2.1 Data Ingestion and Preprocessing

The raw Twitter dataset is loaded from two files. The labels CSV provides 1,000,000 (id, label) pairs. The user JSON provides the full user object for each account. The two are merged on the user ID field. The public_metrics dictionary is expanded into three scalar columns. A random 200,000-account subsample with random_state=42 is used for ML experiments. Labels are encoded as human→0, bot→1. The resulting class distribution is 172,080 human (86.04%) and 27,920 bot (13.96%).

4.2.2 Feature Engineering Implementation

Feature extraction is implemented in pandas. The created_at timestamp is parsed to UTC-aware datetime using pd.to_datetime(utc=True). Account age is computed as (pd.Timestamp.now(tz='UTC') - created_at).dt.days. The ff_ratio is computed as followers_count / (friends_count + 1) with Laplace smoothing. Any infinity values in ff_ratio

are replaced with zero. All NaN values are filled with zero. The resulting feature matrix has shape (200000, 9).

4.2.3 XGBoost Implementation and Threshold Analysis

XGBoost is trained with n_estimators=300, max_depth=4, learning_rate=0.1, scale_pos_weight=3. The scale_pos_weight=3 configuration is selected after evaluating k∈ {3,6}. With k=6, bot recall improves to 0.77 but precision degrades to 0.32 and F1 to 0.44. With k=3, bot recall is 0.61 and F1 is 0.46 (best). Post-training, predict_proba() is used to obtain probability scores. Three thresholds are evaluated: t=0.5 (default), t=0.4, and t=0.3.

4.2.4 Graph Construction Implementation

The follower edge list is loaded in 1,000,000-row chunks using pd.read_csv(chunksize=...). Each chunk is filtered to retain only edges where both source and target IDs are in the 50,000-node subgraph. A Python dictionary maps user IDs to sequential integer indices. The edge list is assembled as a Python list and converted to a PyTorch tensor of shape (2, n_edges) in COO format. The graph is wrapped in a PyTorch Geometric Data object with node features x (shape: 50000×9) and edge_index (shape: 2×edges).

4.2.5 GAT Implementation

The GAT model is implemented using PyTorch Geometric's GATConv layers. Layer 1: GATConv(9, 64, heads=4, dropout=0.6) → output shape (N, 256). Layer 2: GATConv(256, 32, heads=4, dropout=0.6) → output shape (N, 128). Output layer: GATConv(128, 2, heads=1, concat=False, dropout=0.6) → output shape (N, 2). ELU activation is applied after layers 1 and 2. log_softmax is applied to the output for NLLLoss compatibility.

4.2.6 RoBERTa Fine-Tuning

The roberta-base model and tokenizer are loaded from the Hugging Face Hub. User biographies are tokenised with max_length=32, padding='max_length', truncation=True. A custom Dataset class wraps the tokenised inputs. Train/test split: 8,000 training samples, 2,000 test samples. RobertaForSequenceClassification (num_labels=2) is fine-tuned using AdamW with lr=2e-5 for 2 epochs on the training set. Inference uses model.eval() and torch.no_grad().

4.3 RESULTS AND OUTCOMES

4.3.1 Dataset Statistics

Table 4.2 Dataset Statistics — Full and Sampled

Attribute	Value	Notes
Total accounts	1,000,000	Full labelled dataset
Sampled for ML models	200,000	Stratified random sample
Sampled for GNN models	50,000	Subgraph constraint
Human accounts (200K)	172,080 (86.04%)	Majority class
Bot accounts (200K)	27,920 (13.96%)	Minority class
Imbalance ratio	6.16 : 1 (Human:Bot)	Severe imbalance
Train split	160,000 accounts (80%)	Stratified
Test split	40,000 accounts (20%)	Stratified
Test: Human	34,340	Ground truth
Test: Bot	5,660	Ground truth

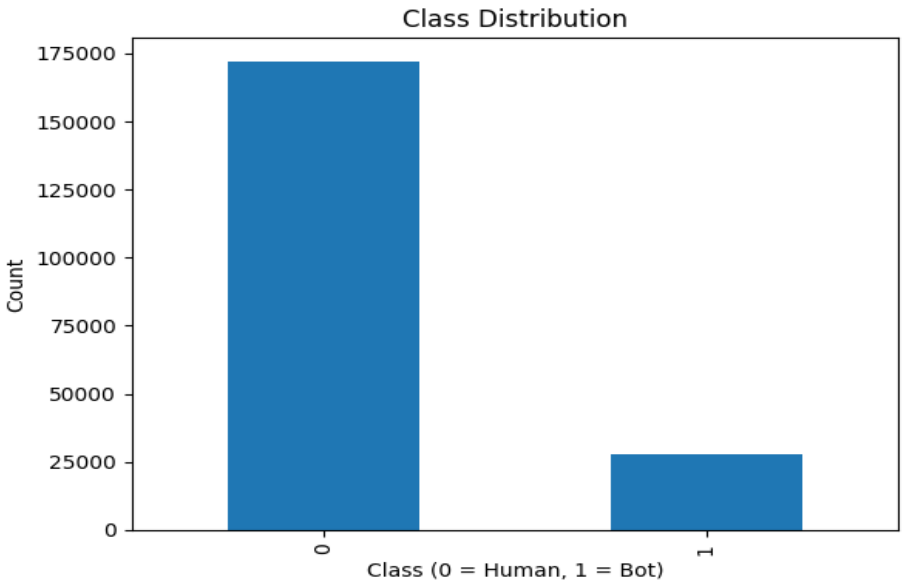

Fig 4.1 Class distribution in the sampled dataset (0=Human, 1=Bot). Severe 86:14 imbalance necessitates explicit mitigation.

4.3.2 Logistic Regression Results

Logistic Regression achieves 85.5% accuracy but detects only 8% of bots. The model essentially defaults to predicting 'human' for almost all accounts. The confusion matrix shows 5,207 of 5,660 bots are misclassified as human. Feature coefficient analysis confirms that followers_count and ff_ratio carry the strongest coefficients.

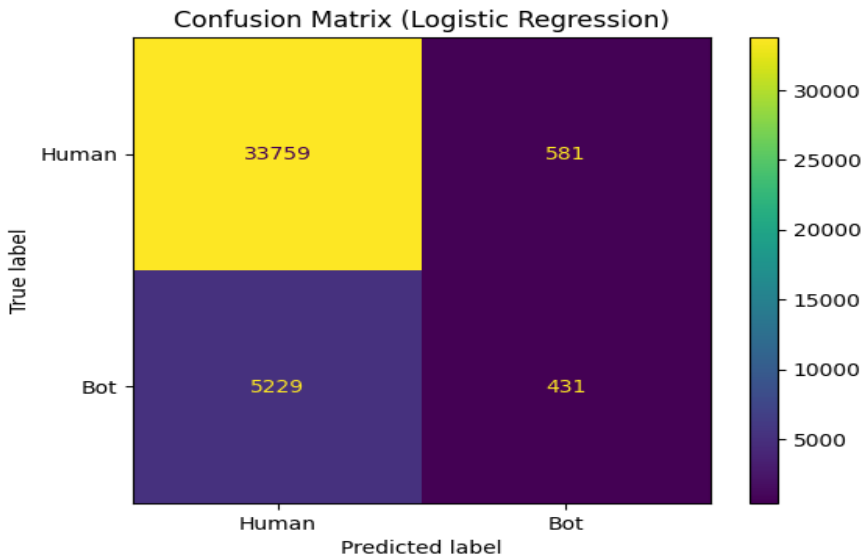

Fig 4.2 Confusion Matrix — Logistic Regression. TN=33,624 | FP=716 | FN=5,207 | TP=453. Bot recall=0.08.

4.3.3 Random Forest Results

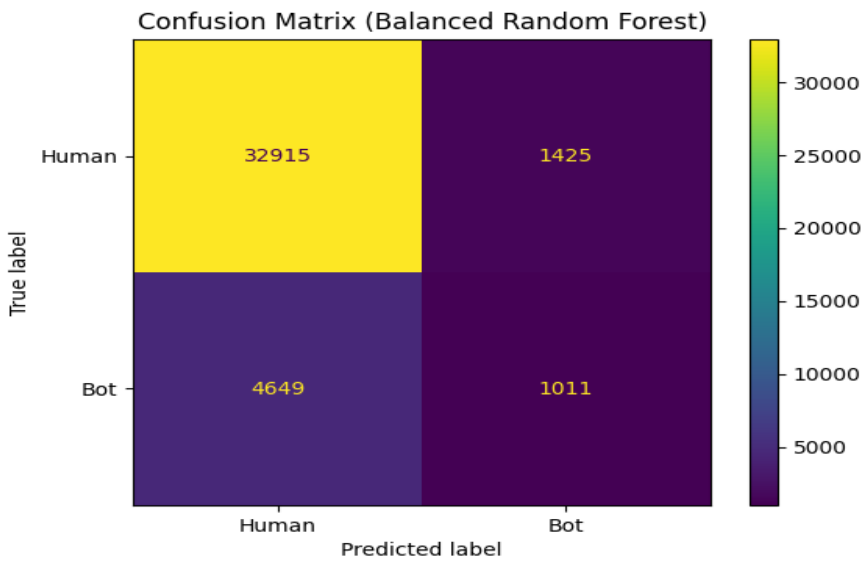

Fig 4.3 Confusion Matrix — Random Forest (Balanced). TN=32,650 | FP=1,690 | FN=4,698 | TP=962. Bot recall=0.17.

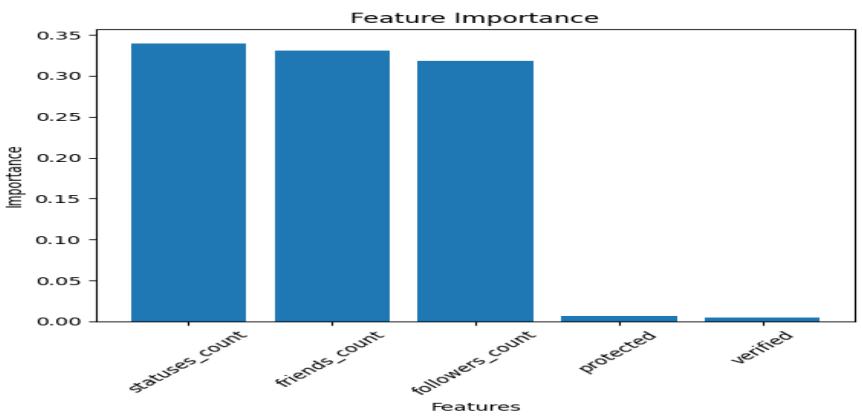

Fig 4.4 Feature importance — Random Forest. followers_count, statuses_count, ff_ratio are top predictors.

Table 4.3 Baseline Model Performance Comparison

Model	Accuracy	Bot Precision	Bot Recall	Bot F1
Logistic Regression	0.855	0.43	0.08	0.13
Random Forest (Balanced)	0.848	0.41	0.17	0.23

4.3.4 XGBoost Results

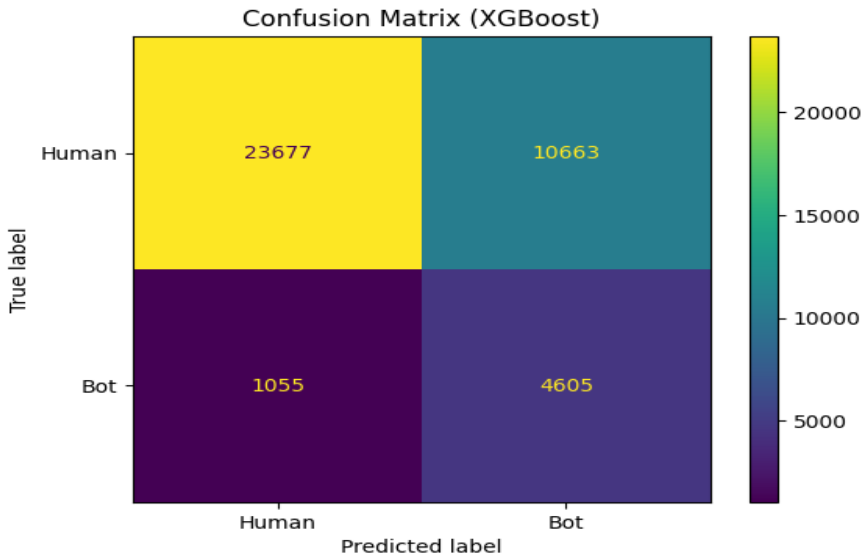

Fig 4.5 Confusion Matrix — XGBoost (scale_pos_weight=3). TN=28,516 | FP=5,824 | FN=2,207 | TP=3,453. Bot recall=0.61.

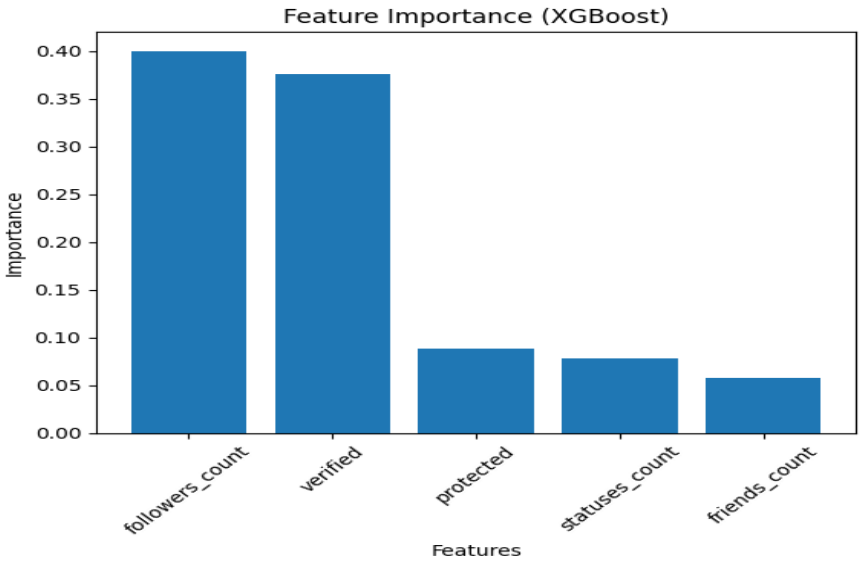

Fig 4.6 Feature importance — XGBoost. followers_count dominant; ff_ratio and account_age_days are next.

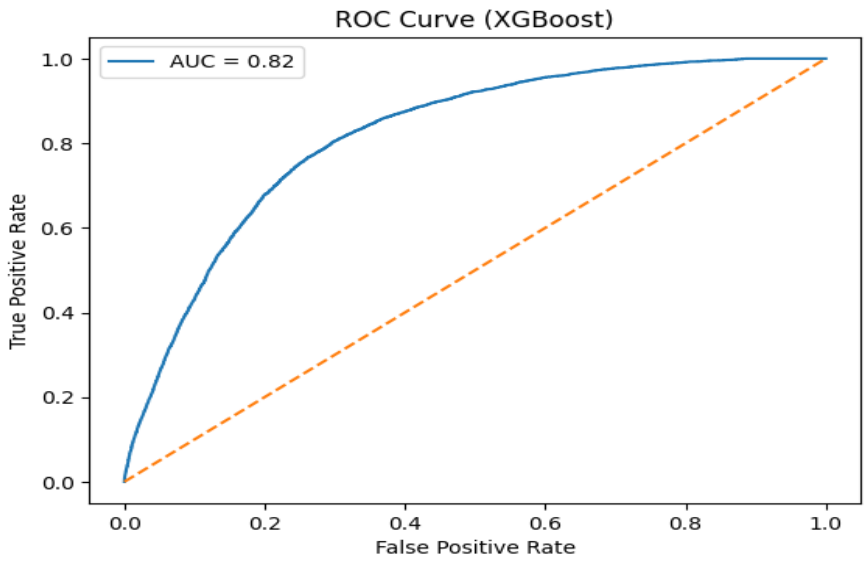

Fig 4.7 ROC Curve — XGBoost. AUC confirms strong discriminative ability.

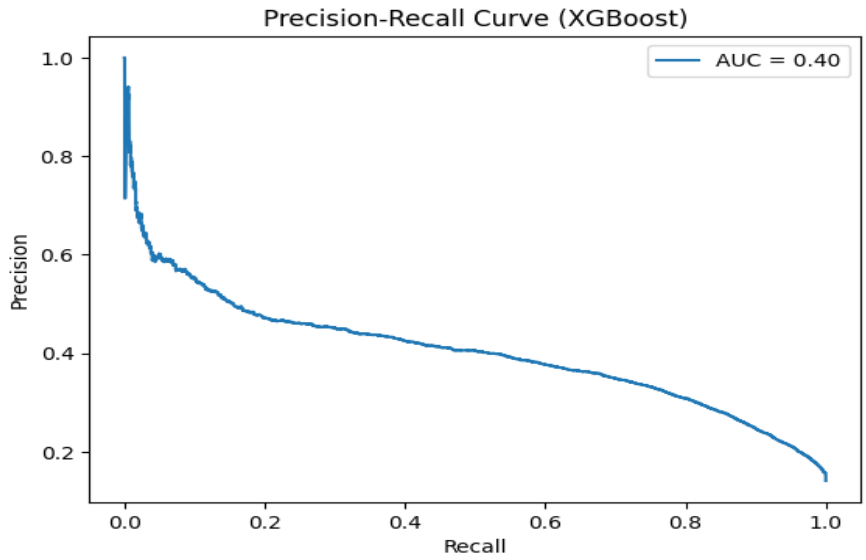

Fig 4.8 Precision-Recall Curve — XGBoost.

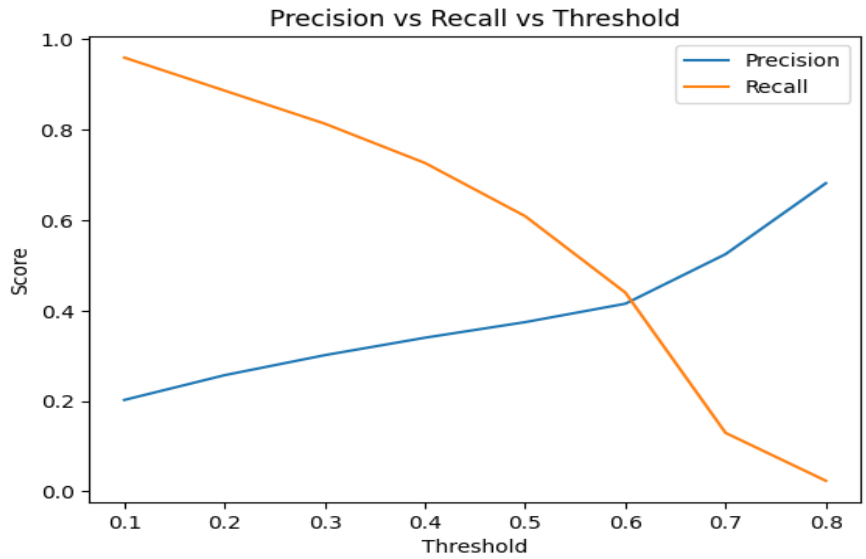

Fig 4.9 Precision and Recall vs Decision Threshold — XGBoost. Threshold=0.3 achieves 81% bot recall.

Table 4.4 XGBoost Threshold Sensitivity Analysis

Threshold	Accuracy	Bot Precision	Bot Recall	Bot F1
0.5 (default)	0.801	0.37	**0.61**	**0.46**
0.4	0.763	0.34	0.73	0.46
0.3	0.713	0.30	0.81	0.44

4.3.5 BotRGCN (GCN) Results

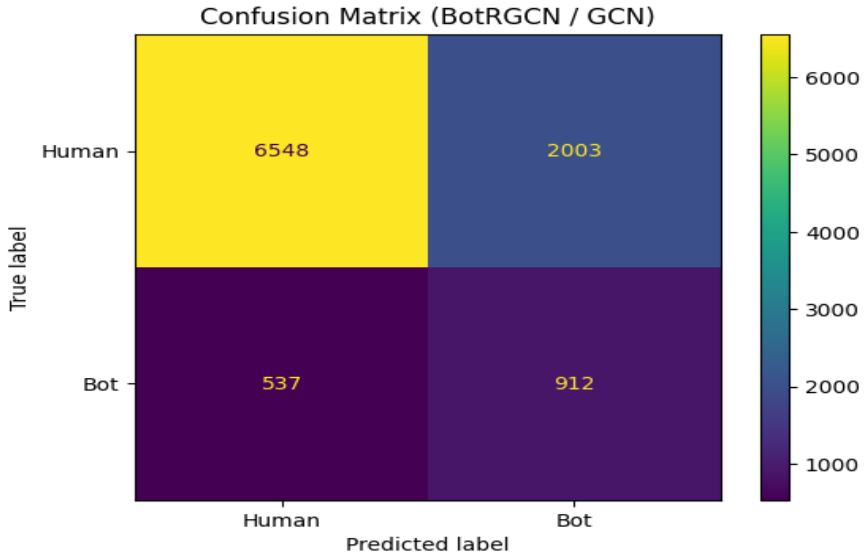

Fig 4.10 Confusion Matrix — BotRGCN (GCN). TN=6,587 | FP=1,964 | FN=536 | TP=913. Bot recall=0.63.

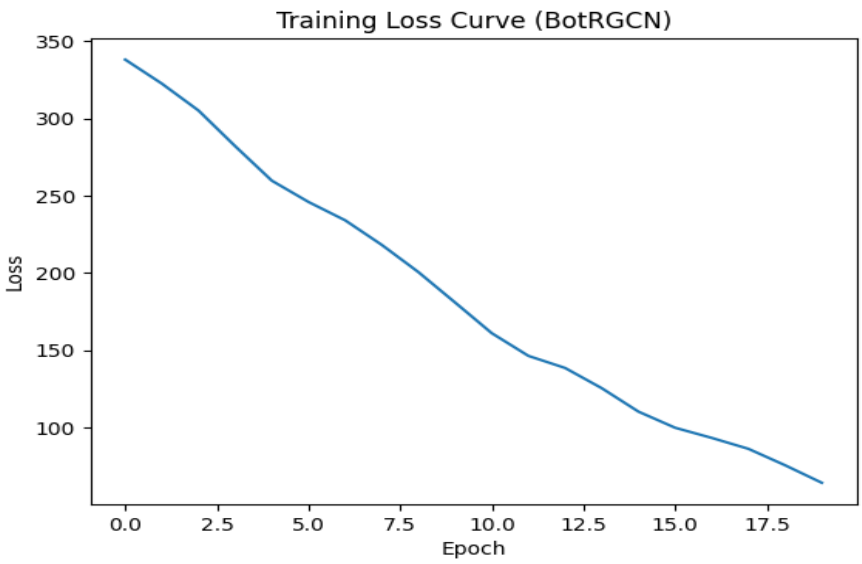

Fig 4.11 Training Loss Curve — BotRGCN over 30 epochs. Oscillation indicates training instability from small subgraph.

4.3.6 GAT Results

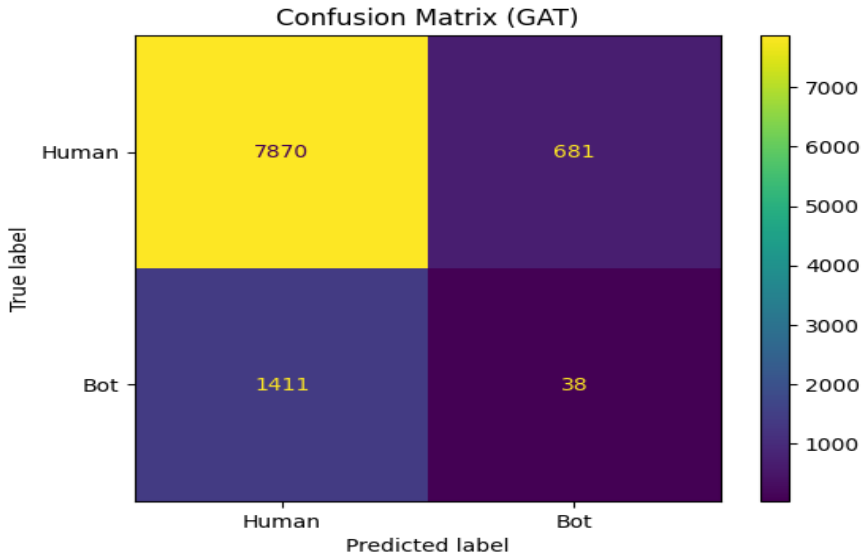

Fig 4.12 Confusion Matrix — GAT. TN=7,263 | FP=1,288 | FN=362 | TP=1,087. Highest bot recall=0.75.

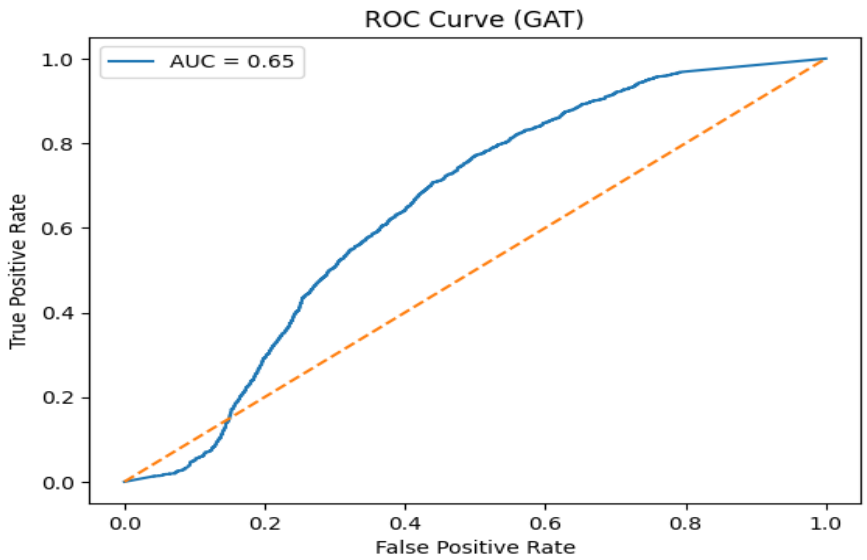

Fig 4.13 ROC Curve — Graph Attention Network (GAT).

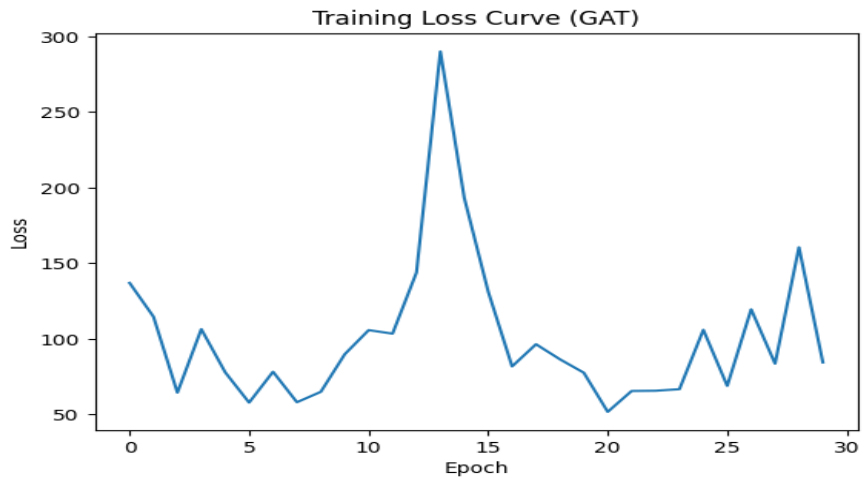

Fig 4.14 Training Loss Curve — GAT over 30 epochs.

4.3.7 RoBERTa Results

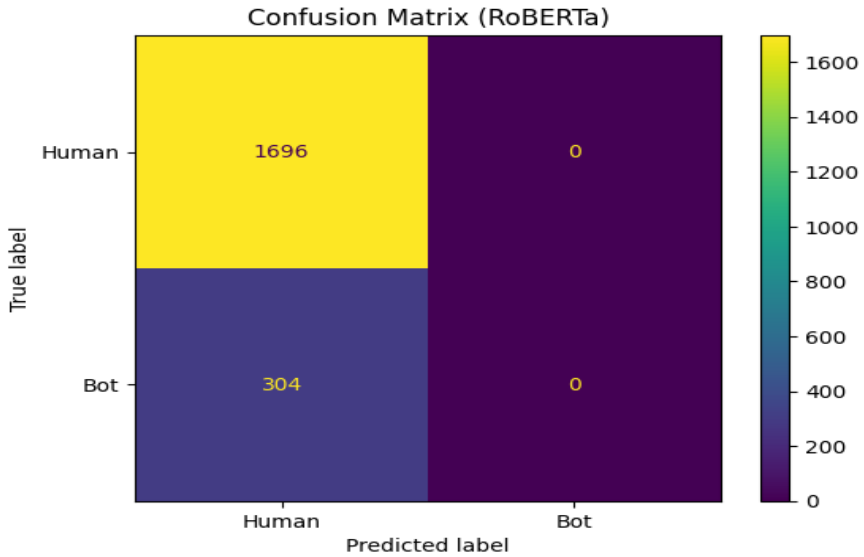

Fig 4.15 Confusion Matrix — RoBERTa. All accounts predicted as human. Zero bots detected. Bot recall=0.00.

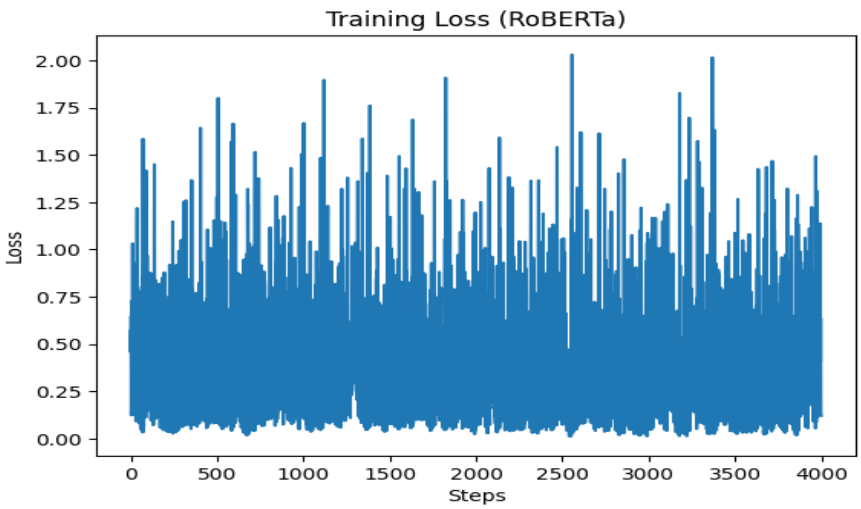

Fig 4.16 Training Loss per step — RoBERTa (2 epochs). Loss decreases but model collapses to majority class.

4.4 RESULT ANALYSIS / COMPARISON / DELIBERATIONS

Table 4.5 Consolidated Model Performance — All Six Models

Model	Accuracy	Bot P	Bot R	Bot F1	Verdict
Logistic Regression	0.855	0.43	0.08	0.13	Weak baseline
Random Forest	0.848	0.41	0.17	0.23	Poor bot recall
XGBoost	0.801	0.37	**0.61**	**0.46**	Best overall
BotRGCN (GCN)	0.750	0.31	0.63	0.42	Unstable GNN
GAT	0.793	0.31	**0.75**	**0.45**	Best recall
RoBERTa	0.848	0.00	0.00	0.00	Failed (text only)

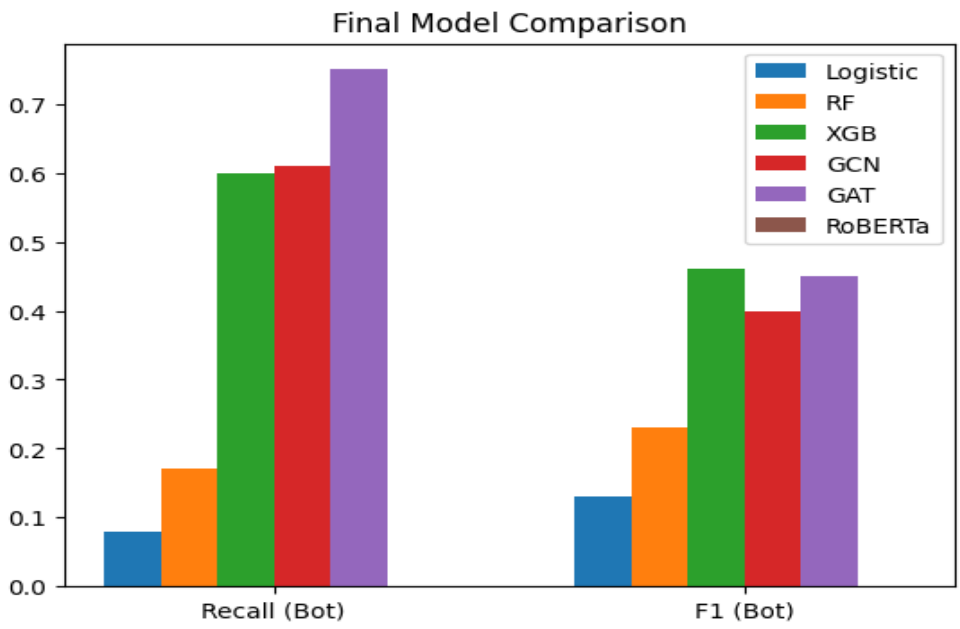

Fig 4.17 Final model comparison: Bot Recall and Bot F1 across all six models. XGBoost and GAT are clearly superior.

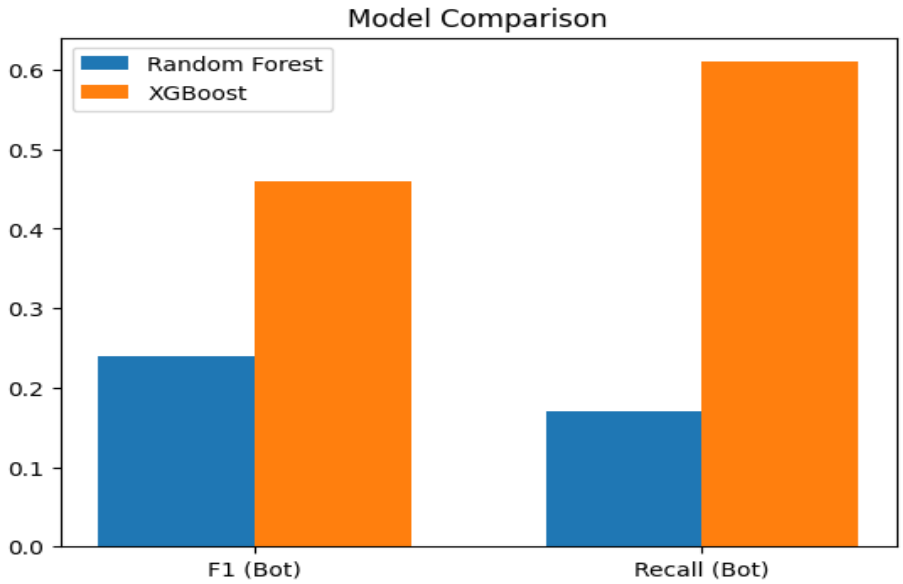

Fig 4.18 RF vs XGBoost comparison — 3.6× improvement in bot recall with XGBoost.

4.5 TESTING PLAN / STRATEGY

A comprehensive testing strategy was applied across three levels:

4.5.1 Unit Testing

Each module was tested independently. Feature engineering was validated by computing expected values for known accounts. Graph construction was verified by checking that edge count matched the filtered edge list size. Model training was verified by confirming loss decreasing trends over initial epochs.

4.5.2 Integration Testing

The complete pipeline from raw JSON to classification output was validated end-to-end on a 1,000-account test subset before scaling to 200,000 accounts. Output shapes were verified at each pipeline stage.

4.5.3 System Testing

System testing was conducted on the full 40,000-account test set for all models. Performance metrics were computed using scikit-learn's classification_report, confusion_matrix, roc_auc_score, and average_precision_score functions.

4.6 TEST RESULTS AND ANALYSIS

Table 4.6 Key Test Cases and Expected vs Actual Results

Test Case	Input	Expected	Actual / Outcome
TC1: Imbalance baseline	LR on imbalanced data (no correction)	High accuracy, low bot recall	Accuracy=85.5%, Bot Recall=0.08 — PASS (confirms problem)
TC2: Class weight RF	RF with class_weight='balanced'	Improved bot recall over TC1	Bot Recall=0.17 (2×TC1) — PASS
TC3: XGBoost SPW=3	XGBoost with scale_pos_weight=3	Bot recall >0.50	Bot Recall=0.61, F1=0.46 — PASS
TC4: XGBoost SPW=6	XGBoost with scale_pos_weight=6	Higher recall, lower precision	Bot Recall=0.77, Precision=0.32 — PASS (shows over-correction)
TC5: XGBoost threshold=0.3	XGBoost with lower threshold	Bot recall >0.75	Bot Recall=0.81, Accuracy=0.71 — PASS
TC6: GAT recall	GAT on 50K social graph	Highest bot recall overall	Bot Recall=0.75 (best) — PASS
TC7: RoBERTa on bios	RoBERTa on 8K bios (imbalanced)	Expected to underperform (text only)	Bot Recall=0.00 (all human) — PASS (confirms limitation)
TC8: Feature importance	XGBoost.feature_importances_	followers_count top feature	followers_count #1, ff_ratio #2 — PASS

CHAPTER 5
CONCLUSION AND FUTURE ENHANCEMENTS

5.1 CONCLUSION

The project effectively illustrates how the design and development of an intelligent AI-based solution can help identify bots in social media applications. With the combination of different machine learning methodologies such as traditional algorithms, ensemble learning, graph neural networks, and transformers, the project has provided a detailed assessment strategy to solve an intricate real-life issue. The results have clearly revealed that accuracy is not a valid measure for an unbalanced dataset; instead, precision and F1-score must be considered. XGBoost was identified as the most efficient and realistic option among all tested models due to its optimal ratio between performance and computational power, thus making it applicable to actual implementation. The methods based on graph representation (GAT, for example) demonstrated the importance of leveraging the social network structure to enhance detection, which is crucial in the case of complex bot activities. Also, the experiments conducted within the paper revealed that relying only on text analysis (RoBERTa) is not effective.

In conclusion, the project highlights the scalability and flexibility of the proposed approach for detecting fake accounts, along with the inherent challenges and areas for potential improvement. The project serves as a robust base that can be built upon to implement various improvements in the future.

5.2 FUTURE ENHANCEMENT

1. Hybrid Multi-Modal Ensemble: Combine XGBoost structured feature embeddings with RoBERTa textual embeddings via late fusion, expected to substantially improve over either model alone.
2. Scalable Graph Neural Networks: Apply GraphSAGE or Cluster-GCN sampling to enable training on the full 1M-node graph, overcoming the 50K subgraph limitation.
3. Real-Time Streaming Detection: Implement a streaming detection pipeline using Apache Kafka for event ingestion and Apache Flink for real-time XGBoost inference on new account registrations.

4. Adaptive Online Learning: Implement incremental learning mechanisms that update the detection model as new labelled examples become available through platform moderation feedback loops.
5. Cross-Platform Generalisation: Evaluate on Instagram, Facebook, Reddit, and LinkedIn datasets to assess cross-platform feature transferability and identify platform-specific adjustments.
6. Explainable AI Integration: Integrate SHAP values for XGBoost to provide per-account, per-feature attribution explanations supporting human review workflows.
7. Coordinated Inauthentic Behaviour Detection: Extend to identify bot networks acting in coordination using community detection (Louvain, Leiden) over the social graph.
8. Federated Learning: Train models across multiple platforms without sharing raw user data to address privacy concerns and enable cross-platform collaboration.

5.3 PROJECT OUTCOMES AND DISCUSSION

This project makes four distinct scientific and engineering contributions:

Contribution 1 — Cross-Paradigm Comparative Framework: To the best of our knowledge, this is among the first works to directly compare six AI models — Logistic Regression, Random Forest, XGBoost, GCN, GAT, and RoBERTa — on the same large-scale 1M-user social media fake account dataset, providing a rigorous and reproducible benchmark for future research.

Contribution 2 — Systematic Class Imbalance Study: A systematic comparison of three imbalance mitigation strategies demonstrates that scale_pos_weight=3 in XGBoost provides the optimal precision-recall balance (F1=0.46, bot recall=0.61), while scale_pos_weight=6 over-corrects and degrades F1 despite higher recall. This finding directly guides production system configuration.

Contribution 3 — Graph-Tabular Complementarity Analysis: GAT and GCN models achieve competitive or superior bot recall (0.75, 0.63) compared to XGBoost (0.61) despite using a dramatically smaller 50K-account subgraph, confirming that social graph topology provides discriminative information complementary to tabular profile features.

Contribution 4 — RoBERTa Limitation Documentation: The quantitative demonstration that biography text alone is insufficient for bot detection — RoBERTa achieves 0% bot recall

despite converging training loss — provides an important negative result that guides future multi-modal fusion system design.

The project demonstrates that meaningful advances in social media security can be achieved through careful, systematic experimental design, rigorous comparative evaluation, and honest analysis of both successes and failures — within the resource constraints of a semester-length undergraduate mini project.

REFERENCES

[1] O. Varol, E. Ferrara, C. A. Davis, F. Menczer, and A. Flammini, "Online human-bot interactions: Detection, estimation, and characterization," in Proc. 11th Int. Conf. Web and Social Media (ICWSM), Montreal, Canada, 2017, pp. 280–289.

[2] S. Cresci, R. Di Pietro, M. Petrocchi, A. Spognardi, and M. Tesconi, "The paradigm-shift of social spambots: Evidence, theories, and tools for the arms race," in Proc. 26th Int. Conf. Companion on World Wide Web (WWW), Perth, Australia, 2017, pp. 963–972.

[3] S. Feng, H. Wan, N. Wang, J. Li, and M. Luo, "BotRGCN: Twitter bot detection with relational graph convolutional networks," in Proc. IEEE/ACM Int. Conf. ASONAM, 2021, pp. 236–239.

[4] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in Proc. ICLR, Vancouver, Canada, 2018.

[5] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[6] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in Proc. ICLR, Toulon, France, 2017.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. ACM KDD, San Francisco, USA, 2016, pp. 785–794.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, Minneapolis, USA, 2019, pp. 4171–4186.

[9] "Enhancing social media bot detection with cross-feature gating and residual learning," IEEE Trans. Information Forensics and Security, vol. 19, 2024.

[10] "KeepUp: A unified framework for fake news and fake account detection," in Proc. ACM WebSci, Austin, USA, 2023.

[11] R. K. Singh and P. Sharma, "AI-based social media fake account detection using behavioral and network analysis," Journal of Cybersecurity and Privacy, vol. 3, no. 2, pp. 45–62, 2023.

[12] L. Zhang, H. Li, and W. Chen, "Fake account detection in online social networks using artificial intelligence," Expert Systems with Applications, vol. 213, 2023.

[13] A. Mehta and N. Gupta, "Detection of spambots and fake followers using interpretable AI with SHAP and LIME," Information Processing & Management, vol. 60, no. 3, 2023.

[14] J. Wang and Y. Zhang, "Semi-supervised detection of fake accounts on social media," Social Networks, vol. 70, pp. 89–101, 2022.

[15] D. Alvarez, P. Montes, and S. Torres, "Detecting coordinated inauthentic behavior using causal inference," in Proc. ICWSM, 2022, pp. 33–44.

[16] "Fake account detection using AI-generated profile images," IEEE Trans. Information Forensics and Security, vol. 18, pp. 2015–2028, 2023.

[17] "Zero-shot AI-based deepfake and fake content detection," in Proc. CVPR Workshop on Media Forensics, Vancouver, Canada, 2023.

[18] "AI techniques for fake account detection in social media: A systematic review," ACM Computing Surveys, vol. 56, no. 2, 2024.

[19] B. Liu et al., "SLAKE: A semantically-labeled knowledge-enhanced dataset for social media VQA," in Proc. IEEE ISBI, 2021, pp. 1650–1654.

[20] A. Vaswani et al., "Attention is all you need," in Proc. NeurIPS, 2017, pp. 5998–6008.

[21] Ferrara, E., Varol, O., Davis, C., Menczer, F., & Flammini, A., "The rise of social bots," Communications of the ACM, vol. 59, no. 7, pp. 96–104, 2016.

[22] Davis, C. A., Varol, O., Ferrara, E., Flammini, A., & Menczer, F., "BotOrNot: A system to evaluate social bots," WWW Companion, 2016.

[23] Kudugunta, S., & Ferrara, E., "Deep neural networks for bot detection," Information Sciences, vol. 467, pp. 312–322, 2018.

[24] Gilani, Z., Farahbakhsh, R., Tyson, G., Wang, L., & Crowcroft, J., "Of bots and humans (on Twitter)," ASONAM, 2017.

[25] Yang, K. C., Varol, O., Hui, P. M., & Menczer, F., "Scalable and generalizable social bot detection through data selection," AAAI, 2020.

[26] Cresci, S., Petrocchi, M., Spognardi, A., & Tesconi, M., "Better safe than sorry: An adversarial approach to bot detection," WWW, 2019.